Chapter 2 Exam Notes: Processes, Threads, IPC, and Scheduling

Modern Operating Systems Study Notes

Contents

1 Scope

Exam focus: Nothing should be skipped. Sections 2.3 and 2.4 are especially important. Be ready for scenario-based questions asking why/where to use mutexes, semaphores, monitors, and scheduling algorithms, including code examples.

2 Processes

2.1 Definition

A **process** is a program in execution. It includes the program code, current CPU state, address space, stack, heap, open files, and OS bookkeeping.

2.2 Process States

- **Running**: currently using the CPU.
- **Ready**: able to run but waiting for CPU.
- **Blocked**: waiting for an event such as I/O, lock release, or message.

2.3 Process Control Block

Typical PCB fields: process state, registers, program counter, stack pointer, scheduling priority, memory-map information, open files, accounting information, parent/child relation, and signal state.

3 Threads

3.1 Process vs Thread

- Processes have separate address spaces.
- Threads in the same process share address space, global variables, heap, and open files.
- Each thread still has its own registers, stack, and execution state.

3.2 Why Threads

- Parallelism on multicore systems.
- Better responsiveness when one activity blocks.
- Natural structure for servers, GUI programs, and pipelines.
- Lower creation/context-switch cost than full processes.

4 2.3 Interprocess Communication

4.1 Race Condition

A **race condition** occurs when the result depends on the timing/interleaving of concurrent operations on shared data.

4.2 Critical Region

A **critical region** is a part of code that accesses shared data and must not be executed by more than one conflicting process/thread at the same time.

Correct critical-region solution:

1. Mutual exclusion: only one inside at a time.
2. No assumptions about CPU speed/count.
3. A process outside the critical region must not block others.
4. No indefinite waiting.

4.3 Mutex

A **mutex** is used for mutual exclusion around a critical section. Use it when the shared resource has exactly one logical owner at a time.

Scenario: shared counter

```
mutex lock;
int counter = 0;

worker() {
    mutex_lock(lock);
    counter = counter + 1;
    mutex_unlock(lock);
}
```

Use mutex when:

- Updating shared variables.
- Modifying shared data structures.
- Protecting a non-reentrant library section.
- Ensuring check-then-act is atomic.

4.4 Semaphore

A **semaphore** is an integer synchronization object manipulated atomically by `down`/`up` or `wait`/`signal`.

- **Binary semaphore**: value 0 or 1; can act like a mutex.
- **Counting semaphore**: counts available units of a resource.

Producer-consumer with bounded buffer

```
semaphore empty = N;
semaphore full = 0;
mutex m;

producer(item x) {
    down(empty);       // wait for free slot
    mutex_lock(m);
    insert(x);
    mutex_unlock(m);
    up(full);          // one more full slot
}

consumer() {
    down(full);        // wait for item
    mutex_lock(m);
    item x = remove();
    mutex_unlock(m);
    up(empty);         // one more free slot
    consume(x);
}
```

Where to use semaphore rather than only mutex:

- Waiting for a condition: buffer nonempty, slots available, resource count available.
- Controlling access to N identical resources.
- Ordering events between threads.

Common mistake: Do not hold a mutex while blocking on a semaphore condition that requires another thread to acquire the same mutex. That pattern often deadlocks.

4.5 Monitor

A **monitor** combines shared data, procedures, mutual exclusion, and condition variables in one language-level abstraction.

```
monitor Buffer {
    condition notFull, notEmpty;
    int count = 0;

    insert(item x) {
        if (count == N) wait(notFull);
        add(x); count++;
        signal(notEmpty);
    }

    remove() {
        if (count == 0) wait(notEmpty);
        item x = take(); count--;
        signal(notFull);
        return x;
    }
}
```

4.6 Message Passing

Processes communicate by `send` and `receive`. Useful when processes do not share memory or run on different machines.

4.7 Barrier

A barrier blocks all participating threads until every thread reaches the barrier. Use in parallel algorithms with phases.

5 Deadlock in Synchronization

Deadlock can happen when each participant waits for another while holding something the other needs. Typical prevention: lock ordering, acquire all resources before use, timeouts, or higher-level monitors.

6 2.4 Scheduling

6.1 Scheduling Goals

- **Batch**: throughput, turnaround time, CPU utilization.
- **Interactive**: response time, fairness.
- **Real-time**: meeting deadlines and predictability.

6.2 Batch Scheduling

FCFS Simple, nonpreemptive, can cause convoy effect.

Shortest Job First Optimal average turnaround if run times are known.

Shortest Remaining Time Next Preemptive version of shortest-job-first.

6.3 Interactive Scheduling

Round Robin Each process gets a time quantum. Small quantum improves response but increases context-switch overhead.

Priority Scheduling Higher-priority processes run first. Must prevent starvation by aging.

Multiple Queues Separate queues for classes of jobs.

Shortest Process Next Estimates future CPU burst from previous behavior.

Lottery Scheduling Randomized proportional-share scheduling.

Fair-Share Scheduling Allocates CPU fairly among users/groups, not just processes.

6.4 Real-Time Scheduling

- **Periodic events**: occur regularly.
- **Aperiodic events**: occur unpredictably.
- A real-time system is schedulable if CPU demand can be met before deadlines.

Math note: For periodic real-time tasks, a basic feasibility check is total utilization: $\sum C_i/P_i \leq 1$, where C_i is CPU time per period and P_i is period. This is necessary for one CPU; specific algorithms may need stricter bounds.

6.5 Policy vs Mechanism

- **Mechanism**: what the kernel can do, e.g., priority levels.
- **Policy**: decision rule, e.g., which process should get high priority.

7 Math and Problem Templates

7.1 CPU Utilization with I/O Wait

If a process waits for I/O fraction p of the time and there are n independent processes, probability all are waiting is p^n. CPU utilization:

$$U = 1 - p^n$$

7.2 Round-Robin Quantum

$$\text{overhead fraction} = \frac{\text{context switch time}}{\text{quantum} + \text{context switch time}}$$

Large quantum behaves like FCFS; very small quantum wastes CPU on switching.

7.3 Turnaround and Waiting Time

$$\text{turnaround} = \text{finish time} - \text{arrival time}$$

$$\text{waiting} = \text{turnaround} - \text{CPU burst time}$$

8 Likely Scenario Questions with Answers

Question

Q1. Shared counter: what should be used?

Answer

Use a mutex because incrementing or decrementing a shared counter is a critical section. Without mutual exclusion, two threads can read the same old value and overwrite each other's update.

```
lock(mutex); counter = counter + 1; unlock(mutex);
```

Use a mutex when exactly one thread may access or update shared state at a time.

Question

Q2. Bounded buffer: what synchronization is needed?

Answer

Use two counting semaphores and one mutex. The `empty` semaphore counts empty slots, `full` counts filled slots, and the mutex protects the buffer data structure.

Producer: $down(empty)$; $down(mutex)$; $insert()$; $up(mutex)$; $up(full)$
Consumer: $down(full)$; $down(mutex)$; $remove()$; $up(mutex)$; $up(empty)$

The semaphores handle ordering; the mutex handles mutual exclusion.

Question

Q3. Many readers but one writer: what should be used?

Answer

Use a readers-writers lock or monitor. Multiple readers can enter together because reading does not change the shared data. A writer must have exclusive access because writing can conflict with both readers and other writers.

Question

Q4. Phase-based parallel computation: what should be used?

Answer

Use a barrier. A barrier blocks each thread at the end of a phase until all participating threads arrive. After the last thread arrives, all are released to the next phase. This is useful when phase $i + 1$ must not start before every thread has completed phase i.

Question

Q5. Processes on different machines: what IPC method is suitable?

Answer

Use message passing because shared memory normally works only between processes that can map common memory on the same machine. Message passing sends explicit messages through the OS or network and is natural for distributed systems.

Question

Q6. Starvation under priority scheduling: how can it be reduced?

Answer

Use aging. Aging gradually increases the priority of a process the longer it waits. This prevents low-priority processes from being postponed forever by newly arriving high-priority processes.

Question

Q7. How should a round-robin quantum be chosen?

Answer

The quantum should be large compared with context-switch time, but not so large that response time becomes poor. If the quantum is too small, the CPU wastes time switching. If it is too large, round robin behaves like FCFS and interactive response becomes worse.